21. What is Container Runtime?

Professional Answer

The container runtime is responsible for pulling images, creating containers, starting, and stopping them according to Pod specifications.

Unprofessional Answer

Docker runs containers.

What is a container runtime in Kubernetes?

The container runtime is responsible for running containers on worker nodes.
It pulls container images and starts, stops, and manages container processes.
Kubernetes communicates with the runtime using the Container Runtime Interface (CRI).
Common runtimes include containerd and CRI-O.
This abstraction allows Kubernetes to be independent of any specific runtime.

Container Runtime (CRI Internals)

What it is

- Implements **Container Runtime Interface**
- Examples:
    - containerd
    - CRI-O

Internal Steps

1. Pull image
2. Create container
3. Create namespaces & cgroups
4. Start container process
5. Monitor container lifecycle

Why CRI matters

Kubernetes is runtime-agnostic

Nice — **container runtime** is a *classic trap topic* in Kubernetes interviews 😄
People say “Docker” and lose points. Let’s do this **properly, deep, and production-grade**.

🧱 Container Runtime – Kubernetes Workhorse

1. What is a container runtime?

A container runtime is the **software responsible for running containers on a node**.
It pulls images, creates containers, starts and stops them.
It manages namespaces, cgroups, and filesystem isolation.
Kubernetes does **not** talk to the runtime directly.
It communicates through **CRI (Container Runtime Interface)**.

2. Why container runtime is important

- Executes Pods on worker nodes
- Enforces resource limits
- Handles container lifecycle
- Interfaces with OS kernel
- Without it, Pods cannot run

👉 Runtime = execution engine.

3. CRI (Container Runtime Interface)

CRI is a **standard API** defined by Kubernetes.
It allows Kubernetes to support **multiple runtimes**.
kubelet uses CRI to talk to the runtime.
This removed Kubernetes’ dependency on Docker.
CRI enables pluggable runtimes.

4. kubelet ↔ container runtime flow

1. kubelet receives Pod spec
2. kubelet calls CRI
3. Runtime pulls image
4. Runtime creates container
5. Container starts

---

6. Status returned to kubelet

👉 kubelet never executes containers itself.

5. Popular container runtimes

1️⃣ containerd (Most common)

- CNCF project
- Used by Docker internally
- Lightweight
- Production standard

2️⃣ CRI-O

- Kubernetes-native runtime
- Used by OpenShift
- Minimal and secure

3 Docker (Deprecated)

- Removed in Kubernetes 1.24+
- Docker Shim removed
- Still used as image builder

---

6. Docker vs containerd (Interview favorite)

Docker	containerd
Full platform	Runtime only
CLI + build	No build
Deprecated in K8s	Native
Heavy	Lightweight

👉 Kubernetes never removed containers — only Docker **shim**.

7. What happens if container runtime goes down?

- Running containers keep running
- New Pods fail to start

- kubelet reports errors
- Node becomes NotReady
- Requires runtime restart

8. Runtime vs Image Registry (Common confusion)

Runtime	Registry
Runs containers	Stores images
On node	Remote
Executes workloads	Supplies images

---

9. How resource limits are enforced

- cgroups control CPU & memory
- Namespaces isolate processes
- Runtime configures kernel settings
- OOM kills enforced at runtime

10. Runtime security features

- Seccomp profiles
- AppArmor / SELinux
- Read-only root filesystem
- User namespaces

👉 Security is runtime + kernel combined.

11. Real production outage example

Incident

- containerd crashed due to disk corruption

Symptoms

- Pods stuck in ContainerCreating
- kubelet errors

- Node NotReady

Fix

- Restarted containerd
- Cleaned runtime state
- Rejoined node

12. Troubleshooting container runtime

systemctl status containerd

journalctl -u containerd

crictl ps

crictl logs <container-id>

Check:

- Disk space
- Image pull errors
- Permission issues

13. Ultra-hard interview question

Q:
Pods stuck in ContainerCreating, API Server healthy. What's wrong?

Answer:
Container runtime issue.
kubelet can't create containers.
Check containerd/CRI-O logs.
Most likely disk or runtime crash.

---

14. Professional vs Unprofessional answer

Unprofessional:

Docker runs containers in Kubernetes.

Professional:

Kubernetes uses CRI-compatible container runtimes like containerd or CRI-O to execute containers on worker nodes, with kubelet acting as the orchestration agent.

15. Exam one-liner

A container runtime executes containers on a node and enforces isolation and resource limits using the OS kernel.

---

16. Senior-level red flags

🚩 "Kubernetes needs Docker"
🚩 "Runtime schedules Pods"
🚩 "Runtime talks to API Server"

(All wrong.)

17. Golden rule

If Pods won't start but scheduling succeeded, blame the container runtime.

---

18. Quick memory trick

- Scheduler → picks node
- Controller → creates Pod
- kubelet → asks runtime
- Runtime → runs container

Perfect 🔥 — **CRI internals** is *elite-level Kubernetes knowledge*.
If you explain this cleanly, interviewers know you've worked **inside** clusters, not just with YAML.

I'll go **deep, structured, exam-ready, and production-real**.

🧠 **CRI (Container Runtime Interface) – Internals Deep Dive**

1. What exactly is CRI?

CRI is a **gRPC-based API** that defines how **kubelet communicates with container runtimes**.
It abstracts container execution away from Kubernetes core.
CRI standardizes lifecycle, image management, and pod sandboxing.
Without CRI, Kubernetes would be tied to one runtime.
CRI enables runtime pluggability.

2. Why CRI was created

Originally, kubelet talked directly to Docker.
This caused tight coupling and complexity.
CRI decoupled Kubernetes from Docker.
It allowed lightweight runtimes like containerd and CRI-O.
Docker Shim was later removed (K8s 1.24).

3. CRI architecture (high level)

kubelet

|

| gRPC (CRI)

|

container runtime (containerd / CRI-O)

|

| OCI

|

runc

|

Linux kernel

👉 CRI is **between kubelet and runtime**, not between runtime and kernel.

4. CRI API categories

CRI defines **two main services**:

1️⃣ RuntimeService

Manages:

- Pod sandboxes
- Containers
- Exec, logs, stats

2️⃣ ImageService

Manages:

- Image pull
- Image list
- Image removal

Both exposed via gRPC.

5. Pod Sandbox (Very Important Concept)

A **Pod sandbox** represents the **Pod-level environment**:

- Network namespace
- IPC namespace
- Shared configuration

Created **once per Pod**.
All containers in a Pod join this sandbox.

👉 Sandbox ≠ Container.

6. Pod creation flow (internals)

1. kubelet receives Pod spec
2. kubelet calls RunPodSandbox()
3. Runtime sets up namespaces & networking
4. kubelet calls CreateContainer()
5. kubelet calls StartContainer()
6. Pod status returned to API Server

This order is critical in exams.

7. CRI runtime responsibilities

- Pull images
- Create sandboxes
- Configure cgroups
- Set security context
- Launch containers

- Report status

kubelet does **none** of this itself.

8. OCI vs CRI (common confusion)

CRI	OCI
K8s-specific	Industry standard
kubelet ↔ runtime	runtime ↔ kernel
gRPC API	Spec + runtime
Pod aware	Container aware

CRI uses **OCI runtimes** like runc.

9. How networking works in CRI

- Runtime creates sandbox
- CNI plugin invoked
- Network namespace created
- IP assigned to Pod
- All containers share it

👉 CRI triggers CNI, not kubelet directly.

10. How storage works in CRI

- Volumes prepared by kubelet
- Runtime mounts volumes into container
- CSI handles backend storage
- Runtime only mounts paths

---

11. Security in CRI

- Applies seccomp profiles
- Enforces AppArmor / SELinux
- Sets user namespaces

- Applies read-only rootfs

SecurityContext is translated by CRI.

12. CRI failure scenario #1

Symptom: Pods stuck in ContainerCreating
Cause: RunPodSandbox() failing
Fix: Check CNI, runtime logs, disk

---

13. CRI failure scenario #2

Symptom: ImagePullBackOff
Cause: ImageService pull failure
Fix: Check registry auth, DNS, runtime logs

14. Tools to inspect CRI

crictl ps

crictl pods

crictl inspect <id>

crictl logs <id>

👉 crictl talks **directly to CRI**, bypassing Kubernetes.

15. containerd CRI internals

containerd has:

- CRI plugin
- Snapshotters (overlayfs)
- Content store
- Task manager

CRI plugin translates kubelet requests.

16. CRI-O internals

- Designed for Kubernetes only

- Minimal daemon
- Direct OCI runtime calls
- No image build features

Used heavily in OpenShift.

17. Why Docker Shim was removed

- Extra abstraction layer
- Performance overhead
- Maintenance burden
- containerd already existed

Kubernetes → CRI → containerd → runc

18. Ultra-hard interview question

Q:
Why is Pod networking created before containers?

Answer:
All containers in a Pod must share the same network namespace.
Network must exist before containers start.
This ensures stable Pod IP.
CRI enforces this via Pod sandbox.

19. Professional vs Unprofessional answer

Unprofessional:

CRI connects Kubernetes and Docker.

Professional:

CRI is a gRPC-based interface that allows kubelet to manage Pod sandboxes and containers using pluggable runtimes like containerd or CRI-O, while delegating low-level execution to OCI runtimes.

---

20. Exam one-liner

CRI enables kubelet to manage containers using any compliant runtime without being runtime-dependent.

21. Senior-level golden rule

If scheduling works but container creation fails, the problem is below kubelet — in CRI or runtime.